
SISTEMA DE BÚSQUEDA Y RESCATE CHAPINWARRIORS

202500286 – Hennsen Armando Rodrigo Aldana Villalobos

Resumen

El proyecto del cual se habla en este documento es un sistema de búsqueda y rescate dirigido a la empresa ChapinWarriors S.A. Este sistema se desarrolla en consola para que sea más ligero y portable entre dispositivos. El sistema brinda una vital asistencia en estrategia y planificación de misiones, facilitando su ejecución.

El proyecto impacta al ámbito social y económico, principalmente por el costo de implementación, pero también por el ahorro en recursos al evaluar el mejor escenario posible tanto para el rescate de civiles como la extracción de recursos. También impacta al ámbito técnico, pues se implementan algoritmos y estructuras de datos necesarias para el correcto funcionamiento.

En Conclusión, el sistema ayuda plenamente al rescate y a la extracción de unidades con las funciones que implementa. El correcto uso de este puede llevar a ahorro económico y mejoras técnicas.

Palabras clave

- Extracción
- Rescate
- Búsqueda
- Algoritmos
- Sistema

Abstract

The project discussed in this document is a search and rescue system aimed at the company ChapinWarriors S.A. This system is developed on a console to make it lighter and more portable between devices. The system provides vital assistance in strategy and mission planning, facilitating its execution.

The project impacts the social and economic sphere, mainly because of the cost of implementation, but also because of the savings in resources by evaluating the best possible scenario for both the rescue of civilians and the extraction of resources. It also impacts the technical field, since algorithms and data structures necessary for proper functioning are implemented.

In conclusion, the system fully helps the rescue and extraction of units with the functions it implements. The correct use of it can lead to economic savings and technical improvements.

Keywords

1. Extraction
2. Rescue
3. Search
4. Algorithms
5. System

Introducción

a. Contexto y planteamiento del problema

El problema abordado en este proyecto puede enmarcarse, desde una perspectiva computacional, dentro de la teoría de grafos aplicada a sistemas de navegación sobre entornos discretizados. La ciudad en conflicto que sobrevuela el dron ChapinEyes se representa como una malla bidimensional de celdas, cada una con un tipo específico (punto de entrada, camino, unidad militar, unidad civil, recurso o celda intransitable). Esta representación equivale, en términos formales, a un grafo no dirigido en el cual cada celda constituye un vértice y cada par de celdas adyacentes (arriba, abajo, izquierda o derecha) constituye una arista. Los algoritmos de recorrido de grafos, como la búsqueda en profundidad (*Depth-First Search*, DFS) o la búsqueda en anchura (*Breadth-First Search*, BFS), son las herramientas estándar para resolver problemas de este tipo, en los cuales se busca determinar si existe un camino entre dos vértices y, de existir, identificar una secuencia válida de pasos (Cormen, Leiserson, Rivest y Stein, 2009).

Lo que distingue al problema planteado de un recorrido de grafo convencional es la introducción de una restricción dinámica: la transitabilidad de una celda no depende únicamente de su tipo, sino también del estado acumulado del agente que la recorre. Una unidad ChapinFighter puede atravesar una celda con unidad militar únicamente si su capacidad de combate, en ese punto específico del recorrido, supera la capacidad de la unidad enemiga —y dicho tránsito reduce su capacidad restante para el resto del camino—. Este matiz convierte al problema en una variante de búsqueda de caminos con estado, en la cual cada nodo del árbol de búsqueda no representa únicamente una posición en la malla, sino una combinación de posición y recurso disponible.

Adicionalmente, el proyecto exige que la solución se sustente sobre estructuras de datos construidas por el propio desarrollador, evitando las colecciones predefinidas del lenguaje de programación. Esta restricción no es meramente académica: obliga a comprender a fondo el funcionamiento interno de listas enlazadas, pilas y colas, y a razonar explícitamente sobre el manejo de memoria dinámica, en contraposición al uso de abstracciones ya optimizadas que ocultan dicha complejidad.

b. Modelado orientado a objetos: diseño de clases

La solución se estructuró siguiendo los principios fundamentales del paradigma orientado a objetos: abstracción, encapsulamiento, herencia y polimorfismo (Booch, 2007). El diseño parte de identificar las entidades del dominio del problema y de establecer relaciones de jerarquía allí donde existe comportamiento compartido pero especializado.

En el núcleo del modelo se encuentra la clase Robot, de la cual heredan ChapinRescue y ChapinFighter. Esta decisión de diseño responde directamente a una necesidad del dominio: ambas unidades comparten atributos comunes (como el nombre) y participan en el mismo flujo general de "ingresar a la ciudad y ejecutar una misión", pero difieren en sus reglas de negocio particulares —únicamente ChapinFighter posee capacidad de combate y puede enfrentar unidades militares—. Modelar esta relación mediante herencia permite que el resto del sistema trate a ambos tipos de robot de forma polimórfica cuando corresponde, sin sacrificar sus comportamientos distintivos.

De manera análoga, la clase Ciudad encapsula la responsabilidad de construir y mantener su propia malla de celdas, delegando en clases especializadas —como el cargador de configuración XML— la tarea de proveerle los datos crudos necesarios, pero conservando internamente el control sobre cómo se

construye y se recorre dicha estructura. Este principio de responsabilidad única evita que la lógica de lectura de archivos se mezcle con la lógica de representación de la malla, facilitando el mantenimiento y las pruebas del sistema.

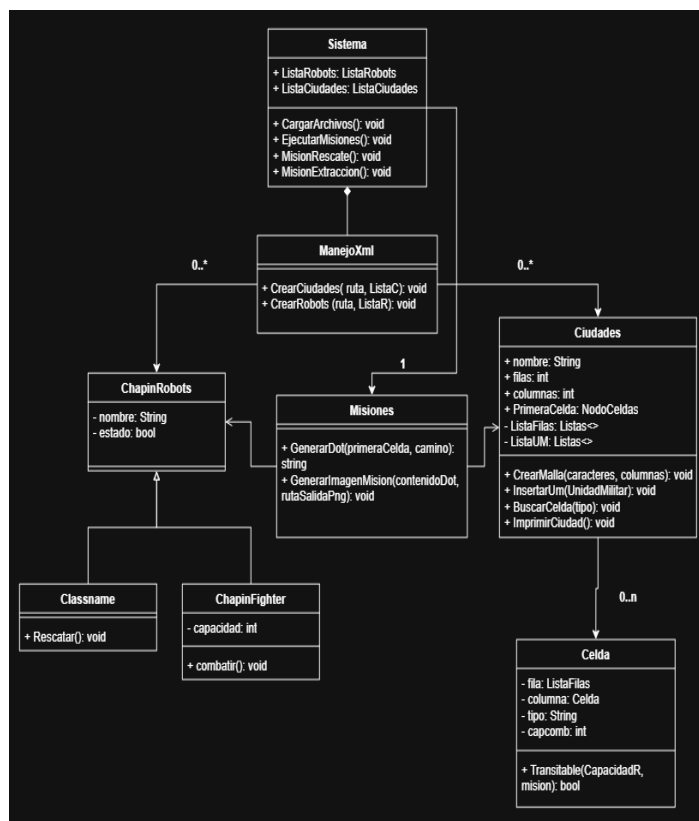


Figura 1. Diagrama de clases general del sistema.
Fuente: elaboración propia.

c. Estructuras de datos propias sobre memoria dinámica

Uno de los retos centrales del proyecto consistió en representar una estructura inherentemente bidimensional —la malla de la ciudad— utilizando exclusivamente memoria dinámica, sin recurrir a arreglos ni a las colecciones genéricas del lenguaje. La solución adoptada consistió en construir una red de nodos enlazados en la que cada *NodoCelda* mantiene cuatro referencias directas hacia sus

vecinos inmediatos: arriba, abajo, izquierda y derecha. Esta decisión, en términos de teoría de estructuras de datos, corresponde a una **lista de adyacencia implícita**, adecuada para grafos dispersos como el que representa una malla rectangular, en la cual cada vértice posee como máximo cuatro conexiones posibles (Cormen et al., 2009). Frente a alternativas como la matriz de adyacencia —cuyo costo en memoria crece de forma cuadrática respecto al número total de celdas—, esta representación resulta considerablemente más eficiente, dado que reserva memoria únicamente para las conexiones que realmente existen.

Sobre esta base se construyeron los Tipos de Dato Abstracto adicionales necesarios para el resto del sistema: una lista simple enlazada genérica, utilizada para almacenar las colecciones de ciudades y robots cargados desde el archivo de configuración; y una pila propia, empleada tanto para la ejecución del algoritmo de búsqueda de caminos como para la reconstrucción del camino encontrado una vez alcanzado el destino.

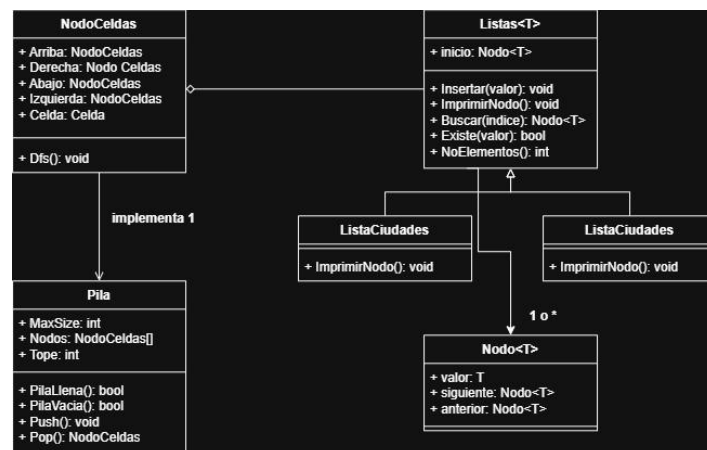


Figura 2. Diagrama de clases de los TDA implementados.
Fuente: elaboración propia.

La tabla I resume las estructuras propias implementadas y su propósito dentro del sistema.

Tabla	I.
Estructuras de datos propias implementadas.	
ESTRUCTURA	PROPÓSITO EN EL SISTEMA
Nodo genérico	Unidad base para listas simples (robots, ciudades)
NodoCelda	Representación de cada celda de la malla, con cuatro punteros a celdas vecinas
Lista simple enlazada	Almacenamiento de la colección de ciudades y de robots configurados
Pila propia	Ejecución del algoritmo de búsqueda de caminos y reconstrucción de la ruta encontrada

Fuente: elaboración propia.

Conviene señalar que, si bien la malla podría haberse representado con una matriz de adyacencia formal, dicha alternativa resultaría inadecuada para este caso particular: al tratarse de un grafo altamente disperso, la proporción de conexiones reales frente a las conexiones posibles es mínima, lo cual desperdiciaría una cantidad considerable de memoria sin aportar ninguna ventaja en tiempo de acceso, dado que cada celda posee, en el peor de los casos, únicamente cuatro vecinos que consultar.

d. Algoritmos de búsqueda de caminos y generación de reportes gráficos

Para la identificación del camino entre el punto de entrada seleccionado y el objetivo de la misión (una unidad civil o un recurso, según corresponda), se implementó un algoritmo de búsqueda en profundidad (DFS) apoyado en la pila propia descrita anteriormente. El algoritmo parte del nodo de entrada, y en cada iteración extrae el nodo superior de

la pila, evalúa si corresponde al destino buscado y, de no ser así, examina sus cuatro vecinos, agregando a la pila aquellos que sean transitables y no hayan sido visitados previamente.

La condición de transitabilidad varía según el tipo de misión. En una misión de rescate, una celda con unidad militar nunca es transitable, independientemente de cualquier otro factor, dado que las unidades ChapinRescue no pueden enfrentar unidades militares. En una misión de extracción, en cambio, la transitabilidad de una celda con unidad militar depende de si la capacidad de combate disponible en ese punto del recorrido supera la capacidad de la unidad enemiga. Esta particularidad exigió que el estado de "capacidad restante" viajara junto con cada rama de exploración del algoritmo, en lugar de tratarse como una variable global compartida entre todas las ramas —de lo contrario, el consumo de capacidad de combate en una rama explorada y luego descartada afectaría incorrectamente a las demás ramas del recorrido—.

Una vez localizado el destino, el camino se reconstruye recorriendo hacia atrás las referencias de nodo padre almacenadas durante la búsqueda, y apilándolas nuevamente para invertir su orden antes de presentarlas. En términos de complejidad computacional, el algoritmo implementado presenta un comportamiento equivalente al de un recorrido DFS estándar sobre un grafo:

$$T(n) = O(V + E) \quad (1)$$

donde:

V = número de vértices (celdas transitables) explorados durante la búsqueda

E = número de aristas (conexiones entre celdas vecinas) evaluadas durante el recorrido

Finalmente, una vez determinado el camino —o confirmada la imposibilidad de la misión—, el sistema genera una representación gráfica de la malla y de la ruta encontrada mediante la herramienta

Graphviz. Dicha herramienta permite describir grafos mediante un lenguaje de descripción textual (formato DOT) y generar automáticamente una representación visual a partir de dicha descripción (Gansner y North, 2000). En el caso particular de este proyecto, la malla se representa forzando una disposición en cuadrícula mediante agrupaciones de rango (*rank=same*) y conexiones invisibles que preservan el orden de filas y columnas, mientras que el camino identificado por el algoritmo se resalta mediante una arista adicional de mayor grosor y color distintivo, superpuesta sobre la estructura base de la malla.

Referencias bibliográficas

Booch, G., Maksimchuk, R. A., Engle, M. W., Young, B. J., Conallen, J., & Houston, K. A. (2007). *Object-oriented analysis and design with applications* (3rd ed.). Addison-Wesley Professional.

Bray, T., Paoli, J., Sperberg-McQueen, C. M.,
Maler, E., & Yergeau, F. (2008). *Extensible Markup
Language (XML) 1.0* (5th ed.). W3C.
<https://www.w3.org/TR/2008/REC-xml-20081126/>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). MIT Press.

Gansner, E. R., & North, S. C. (2000). An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11), 1203–1233.

Microsoft. (s.f.). *Overview - LINQ to XML*.
Microsoft Learn. Recuperado el 27 de agosto de
2026, de [https://learn.microsoft.com/en-
us/dotnet/standard/linq/linq-xml-overview](https://learn.microsoft.com/en-us/dotnet/standard/linq/linq-xml-overview)

Apéndice

Apéndices

Los apéndices que se presentan a continuación complementan el contenido expuesto en el desarrollo del ensayo, incorporando los modelos gráficos completos, ejemplos de datos de prueba y tablas de referencia técnica utilizados durante la construcción de la solución.

Apéndice A. Diagrama de clases completo del sistema

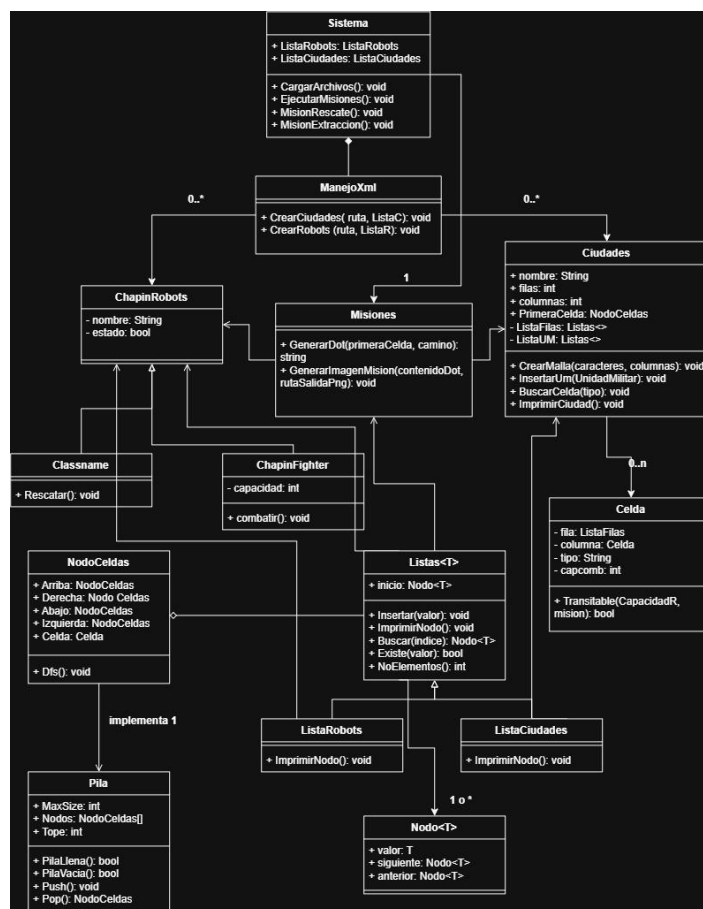


Figura A.1. Diagrama de clases completo del sistema. Fuente: elaboración propia.

Apéndice D. Ejemplo de archivo de configuración XML utilizado para pruebas

A continuación se presenta un archivo de configuración de prueba, construido siguiendo el formato especificado en el enunciado del proyecto, utilizado para validar el funcionamiento de las misiones de rescate y de extracción de recursos durante el desarrollo.

```
<?xml version="1.0"?>
<configuracion>
  <listaCiudades>
    <ciudad>
      <nombre filas="6"
columnas="6">CiudadPrueba</nombre>
      <fila numero="1">"*****"</fila>
      <fila numero="2">"*E  *"</fila>
      <fila numero="3">"*  **  *"</fila>
      <fila numero="4">"*  **  *"</fila>
      <fila numero="5">"*C  R*"</fila>
      <fila numero="6">"*****"</fila>
      <unidadMilitar fila="4"
columna="5">15</unidadMilitar>
    </ciudad>
  </listaCiudades>
  <robots>
    <robot>
      <nombre
tipo="ChapinRescue">robot01</nombre>
    </robot>
    <robot>
      <nombre tipo="ChapinFighter"
capacidad="50">robot02</nombre>
    </robot>
  </robots>
</configuracion>
```

En este caso de prueba, la unidad civil ubicada en la fila 5, columna 2 es alcanzable desde el punto de entrada sin necesidad de atravesar ninguna unidad militar, mientras que el recurso ubicado en la fila 5, columna 5 únicamente es alcanzable venciendo la unidad militar de capacidad 15 ubicada en la fila 4, columna 5.

Apéndice E. Tabla de métodos principales de los Tipos de Dato Abstracto implementados

Tabla E.1. Métodos principales de los TDA propios del sistema.

ESTRUCTURA	MÉTODO	DESCRIPCIÓN
Listas<T>	Insertar(T dato)	Agrega un elemento al final de la lista
	Buscar(T valor)	Retorna el elemento cuyo nombre coincide con el buscado
Listas<T>	Existe(T dato)	Indica si un elemento ya se encuentra en la lista
Pila	Push(NodoCeldas dato)	Agrega un elemento a la parte superior de la pila
	Pop()	Extrae y retorna el elemento superior de la pila
Pila	PilaVacía()	Indica si la pila no contiene elementos
Ciudad	CrearMalla(caracteres, columnas)	Construye la malla de celdas enlazando punteros entre vecinos